

OrderFlow

Monitoring & Observability Task

Educational Backend Project • .NET 8

MAIN GOAL

Make OrderFlow easier to monitor, understand, and troubleshoot. You do not need to build a complicated monitoring system.

Task Objective

The OrderFlow backend is already implemented. Your task is to add basic Monitoring and Observability so that you can understand what is happening inside the application while it is running.

What You Will Build

Area	What to add
Monitoring	Metrics for requests, duration, errors, orders, and worker activity.
Logging	Useful structured logs for important operations and failures.
Tracing	Traces for Create Order, Get Order By ID, and Get Orders.
Health Checks	Application, SQL Server, and Redis health checks.
Dashboard	A simple Grafana dashboard with the required panels.

Area	What to add
Alerts	Two alerts based on operational conditions.

1. Monitoring

Add monitoring for the main application operations. The system should provide information about:

- Number of HTTP requests
- Request duration
- HTTP errors
- Orders created
- Pending orders
- Background Worker activity

Requirement: Choose the appropriate metric type for each metric.

2. Logging

Add useful logs for important operations. At minimum, log:

- Order creation
- Order retrieval
- Cache hit / miss
- Background Worker processing
- Errors

Use suitable log levels such as Information, Warning, and Error.

Logs should contain useful information such as the operation name or Order ID when appropriate.

3. Tracing

Add tracing to important OrderFlow operations. At minimum, trace:

- Create Order
- Get Order By ID
- Get Orders

The trace should help you understand how much time each operation takes and where time is spent.

4. Health Checks

Add health checks for the following components:

Component	Expected purpose
Application	Confirm the API application is running and able to respond.
SQL Server	Confirm the database dependency is available.
Redis	Confirm the cache dependency is available.

The health endpoint should clearly show whether the application and its dependencies are healthy or not.

5. Grafana Dashboard

Create a simple Grafana dashboard containing at least the following panels:

Panel	Purpose
Request Count	Show the volume of incoming requests.
Request Duration	Show how long requests take.
Error Count	Show HTTP/application errors.
Orders Created	Show the number of orders created.
Pending Orders	Show the current pending-order situation.

You may add more panels if you think they are useful.

6. Alerts

Create two simple alerts. Choose any two from the following:

- High Error Rate
- High Request Duration
- Redis Unavailable
- High Pending Orders

Example Alert	Meaning
High Pending Orders	The number of pending orders is increasing and the Background Worker may not be processing normally.

For each alert, write one sentence explaining what the alert means.

7. Simple Test Scenarios

Scenario 1 — Normal Application

- Create some orders.
- Retrieve orders.
- Open Grafana.
- Check your metrics.
- Check the logs.
- Open a trace.
- Verify that the activity is visible.

Scenario 2 — Dependency Failure

Stop either Redis or SQL Server. Then make a request to the API.

Check the following:

- Health Check
- Logs
- Metrics
- Trace

Write a short explanation of what changed.

8. Deliverables

- Source code
- Grafana dashboard screenshot
- Screenshot of a trace
- Screenshot of logs
- Screenshot of health checks
- Two alert configurations

9. Expected Learning Outcome

After completing this task, you should be able to:

- Add basic metrics to a .NET application.
- Add useful structured logs.
- Create traces for important operations.
- Monitor application dependencies.
- Build a simple Grafana dashboard.
- Create basic alerts.
- Use Monitoring and Observability tools to understand what is happening inside an application.

Submission Checklist

Status	Item
☐	Source code completed
☐	Metrics are visible
☐	Required logs are visible
☐	Required traces are visible
☐	Application + SQL Server + Redis health checks work
☐	Grafana dashboard contains the required panels
☐	Two alerts are configured
☐	Normal scenario tested
☐	Dependency failure scenario tested
☐	Required screenshots collected

Remember: The goal is not to build a complicated monitoring system. The goal is to make OrderFlow easier to observe, understand, and troubleshoot.